Evaluating Supply Chain Flexibility of Third-Party Logistics (3PL) Providers Using Linear Algebra Techniques

Sharwan(CS24B1059)
Jyothir(CS24B1058)

December 11, 2025

Abstract

This report presents a detailed framework to evaluate and rank Third-Party Logistics (3PL) providers under multiple operating scenarios (Normal, Peak, Disruption). The project implements a reproducible pipeline that: generates scenario data, normalizes the data, computes weighted scores, examines relationships using covariance and correlation, extracts principal components (PCA), performs Singular Value Decomposition (SVD), and compares provider performance across scenarios. The code base (Python) is modular: data generation, normalization, scoring, analysis and orchestration. This document explains all mathematical steps, the role of each code file, and how to reproduce results and plots.

Contents

1 Introduction

Outsourcing logistics to Third-Party Logistics providers (3PLs) is common. Decision-makers must evaluate providers across multiple criteria (cost, delivery time, flexibility etc.) and under different scenarios. This project applies Linear Algebra tools to create an objective, reproducible evaluation pipeline:

- Represent provider performance as matrices,
- Normalize and weight criteria,
- Score providers by matrix-vector multiplication,
- Use PCA and SVD to reveal latent structure,
- Run scenario analysis to assess flexibility.

2 Problem Statement

Given a set of providers and numerical scores across multiple criteria and scenarios, determine:

1. A fair ranked order of providers for each scenario.
2. How rankings change across scenarios (measure of flexibility).
3. What latent factors or combinations of criteria dominate the performance (via PCA/SVD).
4. A reproducible pipeline and codebase to generate the data, run analyses, and produce plots.

3 Dataset

The dataset is synthetic but realistic: five providers (A–E), six criteria, three scenarios.

- Criteria: Cost (converted to score; higher is better), Time, Flexibility, Reliability, Capacity, Coverage.
- Scenarios: Normal, Peak, Disruption.

Data are stored in a tensor $X \in \mathbb{R}^{P\times C\times S}$ where $P = 5$, $C = 6$, $S = 3$. Each scenario s has an associated matrix $X^{(s)} \in \mathbb{R}^{P\times C}$.

3.1 Example: Normal scenario matrix

For concreteness, the Normal scenario matrix $X^{(\text{Normal})}$ used by the code is:

$$X^{(\text{Normal})} = \begin{bmatrix} 72 & 85 & 60 & 90 & 70 & 75 \\ 58 & 78 & 82 & 86 & 65 & 80 \\ 65 & 80 & 70 & 88 & 75 & 70 \\ 50 & 70 & 90 & 82 & 68 & 60 \\ 68 & 88 & 66 & 91 & 72 & 78 \end{bmatrix}.$$

Rows correspond to providers A, B, C, D, E respectively; columns correspond to the six criteria in the order described.

4 Mathematical Methods and Computations

This section presents every mathematical step used in the code pipeline and what it accomplishes.

4.1 Normalization

Different criteria have different units and ranges. Normalization makes the columns comparable.

Two methods are used:

4.1.1 Min–Max normalization

For a matrix $X \in \mathbb{R}^{P \times C}$ (one scenario), min–max normalizes each column j by:

$$x_{ij}^{(n)} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}.$$

Result: each column scaled to $[0, 1]$. Implemented in `normalizer.py` as:

```
X_min = X.min(axis=0)
X_max = X.max(axis=0)
X_minmax = (X - X_min) / (X_max - X_min)
```

4.1.2 Z-score normalization

For PCA and covariance analysis we use z-scores (centering + scaling):

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the column mean and standard deviation. Implemented with SciPy's `zscore`:

```
from scipy.stats import zscore
X_z = zscore(X, axis=0)
```

4.2 Weighted scoring

We want a single score per provider reflecting business priorities. Let $w \in \mathbb{R}^C$ be a weight vector with $\sum_j w_j = 1$ and $w_j \geq 0$.

Using the min–max normalized matrix X_{norm}, the score vector $S \in \mathbb{R}^P$ is computed by:

$$S = X_{\text{norm}}\, w.$$

Each provider i gets score S_i. In code:

```
w = np.array([0.25,0.20,0.20,0.15,0.10,0.10])
S = X_minmax.dot(w)
```

4.2.1 AHP / Eigenvector weighting (optional)

The Analytic Hierarchy Process derives weights from pairwise criteria comparisons. If $P \in \mathbb{R}^{C\times C}$ is the pairwise matrix (reciprocal), the principal eigenvector of P (normalized) gives weights w. The code includes a geometric-mean approximation if required:

```
def ahp_weights(pairwise_matrix):
    g = np.prod(pairwise_matrix, axis=1) ** (1.0 /
       ↪pairwise_matrix.shape[0])
    w = g / g.sum()
    return w
```

4.3 Covariance and Correlation

For the centered data matrix $X_c = X - \mathbf{1}\mu^\top$ (rows providers, columns criteria), the sample covariance matrix is:

$$C = \frac{1}{P-1} X_c^\top X_c \in \mathbb{R}^{C\times C}.$$

The correlation matrix is derived from C by normalizing each element by the product of standard deviations. In code:

```
C = np.cov(X, rowvar=False)
R = np.corrcoef(X, rowvar=False)
```

Interpretation: large positive covariance/correlation between two criteria suggests redundancy.

4.4 Principal Component Analysis (PCA)

PCA finds orthogonal directions (principal components) capturing maximum variance. For centered data X_c:

$$C = \frac{1}{P-1} X_c^\top X_c = V \Lambda V^\top,$$

where V contains eigenvectors and Λ diagonal eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots$. Project data onto top k eigenvectors V_k:

$$Y = X_c V_k \quad (P \times k).$$

In Python:

```
eigvals, eigvecs = np.linalg.eigh(C)
# sort descending, take top k
Y = Xc @ eigvecs[:, ::-1][:, :k]
```

We show 2D PCA scatter (PC1 vs PC2) for visualization.

4.5 Singular Value Decomposition (SVD)

SVD decomposes any (centered) matrix X_c as:

$$X_c = U \Sigma V^\top,$$

with orthogonal U, orthogonal V and diagonal Σ with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

Relationship to PCA: eigenvectors of $X_c^\top X_c$ are columns of V; $\sigma_i^2/(P-1)$ correspond to PCA eigenvalues. SVD is useful for:

- low-rank denoising (keep top k singular values),
- robust ranking using dominant left singular vector.

In Python:

```
U, Svals, Vt = np.linalg.svd(Xc, full_matrices=False)
```

5 Codebase: files and detailed explanation

All code is in Python and modular. Below we explain each file, its purpose, and include the most important code snippets.

5.1 data_generator.py

Purpose: programmatically create the $5 \times 6 \times 3$ dataset and save a CSV with rows (Provider, Scenario, Criteria...).

```
# data_generator.py (core excerpt)
import numpy as np
import pandas as pd

providers = ['A','B','C','D','E']
scenarios = ['Normal','Peak','Disruption']
X = np.zeros((5,6,3))
# Fill Normal
X[:,:,0] = np.array([[72,85,60,90,70,75],
                     [58,78,82,86,65,80],
                     [65,80,70,88,75,70],
                     [50,70,90,82,68,60],
                     [68,88,66,91,72,78]])
# Fill Peak and Disruption similarly...
rows=[]
for s_idx,s in enumerate(scenarios):
    for p_idx,p in enumerate(providers):
        rows.append([p, s] + X[p_idx,:,s_idx].tolist())
cols=['Provider','Scenario','Cost','Time','Flexibility','
   ↪Reliability','Capacity','Coverage']
pd.DataFrame(rows, columns=cols).to_csv('3PL_dataset.csv',
   ↪index=False)
```

5.2 normalizer.py

Purpose: implement Min–Max and Z-score normalizers and save normalized matrices for each scenario.

```python
# normalizer.py
import numpy as np
from scipy.stats import zscore

def minmax_normalize(X):
    X_min = X.min(axis=0)
    X_max = X.max(axis=0)
    return (X - X_min) / (X_max - X_min)

def zscore_normalize(X):
    return zscore(X, axis=0)
```

5.3 scorer.py

Purpose: compute weighted scores, optional AHP weighting, and save rankings.

```python
# scorer.py
import numpy as np
import pandas as pd

def compute_scores(X_norm, providers, weights):
    S = X_norm.dot(weights)
    df = pd.DataFrame({'Provider': providers, 'Score': S})
    return df
```

5.4 analyzer.py

Purpose: compute covariance, correlation, PCA, SVD and produce plots (PCA scatter, singular value decay, heatmaps). Saves PNGs used in the presentation.

```python
# analyzer.py (core)
import numpy as np
import matplotlib.pyplot as plt

def compute_pca(X_z, providers, outname):
    C = np.cov(X_z, rowvar=False)
    eigvals, eigvecs = np.linalg.eigh(C)
    idx = np.argsort(eigvals)[::-1]
    eigvals = eigvals[idx]
    eigvecs = eigvecs[:,idx]
    scores = X_z @ eigvecs
    var_exp = eigvals / eigvals.sum()
    # plot PC1 vs PC2
    plt.figure(figsize=(6,4))
    plt.scatter(scores[:,0], scores[:,1])
    for i,label in enumerate(providers): plt.text(scores[i,0],
        ↪ scores[i,1], label)
    plt.xlabel(f'PC1 ({var_exp[0]*100:.2f}% var)')
    plt.ylabel(f'PC2 ({var_exp[1]*100:.2f}% var)')
    plt.grid(True, linestyle='--', alpha=0.4)
```

```
    plt.tight_layout()
    plt.savefig(outname, dpi=150)
    plt.close()
    return eigvals, eigvecs, scores, var_exp

def compute_svd(Xc):
    U, Svals, Vt = np.linalg.svd(Xc, full_matrices=False)
    return U, Svals, Vt
```

5.5 main.py

Purpose: orchestrates pipeline for all scenarios.

```
# main.py (core excerpt)
from data_generator import X, providers, scenarios
from normalizer import minmax_normalize, zscore_normalize
from scorer import compute_scores
from analyzer import compute_pca, compute_svd
import numpy as np
import pandas as pd

weights = np.array([0.25,0.20,0.20,0.15,0.10,0.10])

for s_idx, scenario in enumerate(scenarios):
    Xs = X[:,:,s_idx]                  # P x C matrix
    X_minmax = minmax_normalize(Xs)  # Min-max normalized
    df_scores = compute_scores(X_minmax, providers, weights)
    df_scores.to_csv(f'scores_{scenario}.csv', index=False)
    X_z = zscore_normalize(Xs)       # z-score for PCA/SVD
    eigvals, eigvecs, scores, var_exp = compute_pca(X_z,
       ↪providers, f'{scenario}_pca.png')
    Xc = Xs - Xs.mean(axis=0)
    U, Svals, Vt = compute_svd(Xc)
    pd.DataFrame({'singular_value': Svals}).to_csv(f'{scenario
       ↪}_singular_values.csv', index=False)
```

6 Detailed Worked Example (Normal scenario)

This subsection demonstrates exact computations performed on the Normal scenario matrix $X^{(\text{Normal})}$.

6.1 Step 1: Min–Max Normalization

Column-wise minima and maxima for the Normal matrix:

$$\min_j = [50,\ 70,\ 60,\ 82,\ 65,\ 60], \qquad \max_j = [72,\ 88,\ 90,\ 91,\ 75,\ 80].$$

For example, normalized cost for provider A:

$$x^{(n)}_{A,\text{cost}} = \frac{72 - 50}{72 - 50} = 1.0.$$

All entries are computed similarly; the full normalized matrix is saved as `3PL_Normal_minmax.csv` by the script.

6.2 Step 2: Weighted Scoring

With weight vector $w = [0.25, 0.20, 0.20, 0.15, 0.10, 0.10]^\top$ and normalized matrix X_{norm}:

$$S = X_{\text{norm}}\, w.$$

The Python script computes S and saves the per-provider scores as `scores_Normal.csv`.

6.3 Step 3: PCA

Using the z-score normalized data X_z, compute covariance C, eigenvalues and eigenvectors, and PCA coordinates $Y = X_z V$. The script saves `Normal_pca.png` and a CSV of PCA scores.

6.4 Step 4: SVD

Center data $X_c = X - \mu$ and compute SVD

$$X_c = U \Sigma V^\top.$$

The script writes the singular values to `Normal_singular_values.csv`.

7 Results (where to find numeric outputs)

All numeric outputs are generated by running `main.py` and saved to the project folder. Example files:

- `3PL_dataset.csv` – raw dataset
- `scores_Normal.csv` – weighted scores (Normal)
- `Normal_pca.png` – PCA figure (Normal)
- `Normal_singular_values.csv` – SVD singular values (Normal)
- Similar files for Peak and Disruption scenarios.

8 Interpretation and Discussion

- Weighted scores give an immediate ranked list per scenario.
- PCA scatterplots reveal clusters and provider similarity.
- SVD singular values show how many latent factors are important.
- Scenario analysis (compare score CSVs) identifies stable vs fragile providers.

9 How to reproduce everything

1. Install Python packages: `numpy`, `pandas`, `scipy`, `matplotlib`.
2. Run:

```
python data_generator.py
python main.py
```

3. Open the generated CSVs and PNGs for insertion into the report or presentation.
4. Optionally change the weight vector in `scorer.py` and re-run for sensitivity analysis.

Appendix: Code excerpts

(Full code files are in the project folder; the key functions are shown in earlier sections.)

References

- K.L. Choy et al., "Leveraging the supply chain flexibility of third party logistics – Hybrid knowledge-based system approach", Expert Systems with Applications, 2008.
- I.T. Jolliffe, Principal Component Analysis, Springer, 2002.
- G.H. Golub and C.F. Van Loan, Matrix Computations, Johns Hopkins University Press.
- T.L. Saaty, The Analytic Hierarchy Process, McGraw-Hill, 1980.